

Smart Retrofit Kit for Legacy Air Conditioners: FreeRTOS on QEMU

Wei JianNan MC555577

1. Introduction

Legacy air conditioners remain widely deployed in homes and offices, yet they lack modern features such as remote control, intelligent adjustment, and network connectivity. Replacing these units with new smart appliances is costly and environmentally inefficient. Inspired by cost-effective upgrade strategies in other domains—such as retrofit kits for legacy military equipment—this project introduces a **Smart Retrofit Kit** that transforms conventional AC units into smart, connected devices without replacing the entire system.

The proposed solution integrates **FreeRTOS running on an STM32F407 microcontroller**, emulated on **QEMU** for development and validation. This approach ensures portability, real-time performance, and safe resource sharing through robust synchronization mechanisms. The system architecture includes multiple tasks for sensor monitoring, infrared control, network connectivity, and external communication, coordinated using FreeRTOS primitives such as **queues** and **mutexes**.

Key motivations and objectives:

- **Modernization without replacement:** Extend the lifespan of legacy AC units by adding smart capabilities.
- **Real-time control and connectivity:** Enable temperature and humidity monitoring, infrared-based AC control, and Wi-Fi integration.
- **Simulation-driven development:** Use QEMU to emulate Cortex-M hardware and peripherals for rapid testing.
- **Multi-tasking and synchronization:** Implement five coordinated FreeRTOS tasks with deterministic scheduling and inter-task communication.
- **Smart features:** AI-powered temperature suggestions based on real-time weather data, bilingual web interface, and remote access.
- **Optional enhancements:** Virtual networking and Rust-based reimplementation for improved safety and maintainability.

This report details the system architecture, task design, communication strategies, and performance evaluation, demonstrating how embedded systems innovation can deliver cost-effective modernization for legacy appliances.

2. Project Objectives

Primary Objectives

- Real-time Environmental Monitoring: Continuous temperature and humidity sensing using SHT30 sensor.
- Infrared Remote Control: NEC protocol-based air conditioner control via IR transmitter and receiver.
- Wireless Connectivity: Integration of ESP8266 Wi-Fi module for remote access.
- Web-based Interface: User-friendly control panel with bilingual support (Chinese/English).
- Intelligent Control System: AI-powered temperature suggestions based on real-time weather data.
- System Integration: Complete QEMU simulation environment with external bridge communication.

Technical Objectives

- RTOS Implementation: Multi-task real-time system using FreeRTOS.
- Hardware Abstraction: Modular HAL design for peripheral management.
- Communication Protocols: I2C, UART, PWM, TCP/HTTP, JSON data exchange.
- Cross-platform Compatibility: QEMU simulation with real hardware portability.
- Performance Optimization: Efficient task scheduling and resource management.

3. System Architecture

The system adopts a **multi-layer architecture** combining embedded firmware, a Python-based bridge server, and a web interface for remote control. This design ensures modularity, scalability, and cross-platform compatibility.

3.1 High-Level Architecture

The system architecture consists of three main layers: **Web Interface**, **Bridge Server**, and **Embedded Firmware running on QEMU**, with explicit data flows from sensors and actuators.

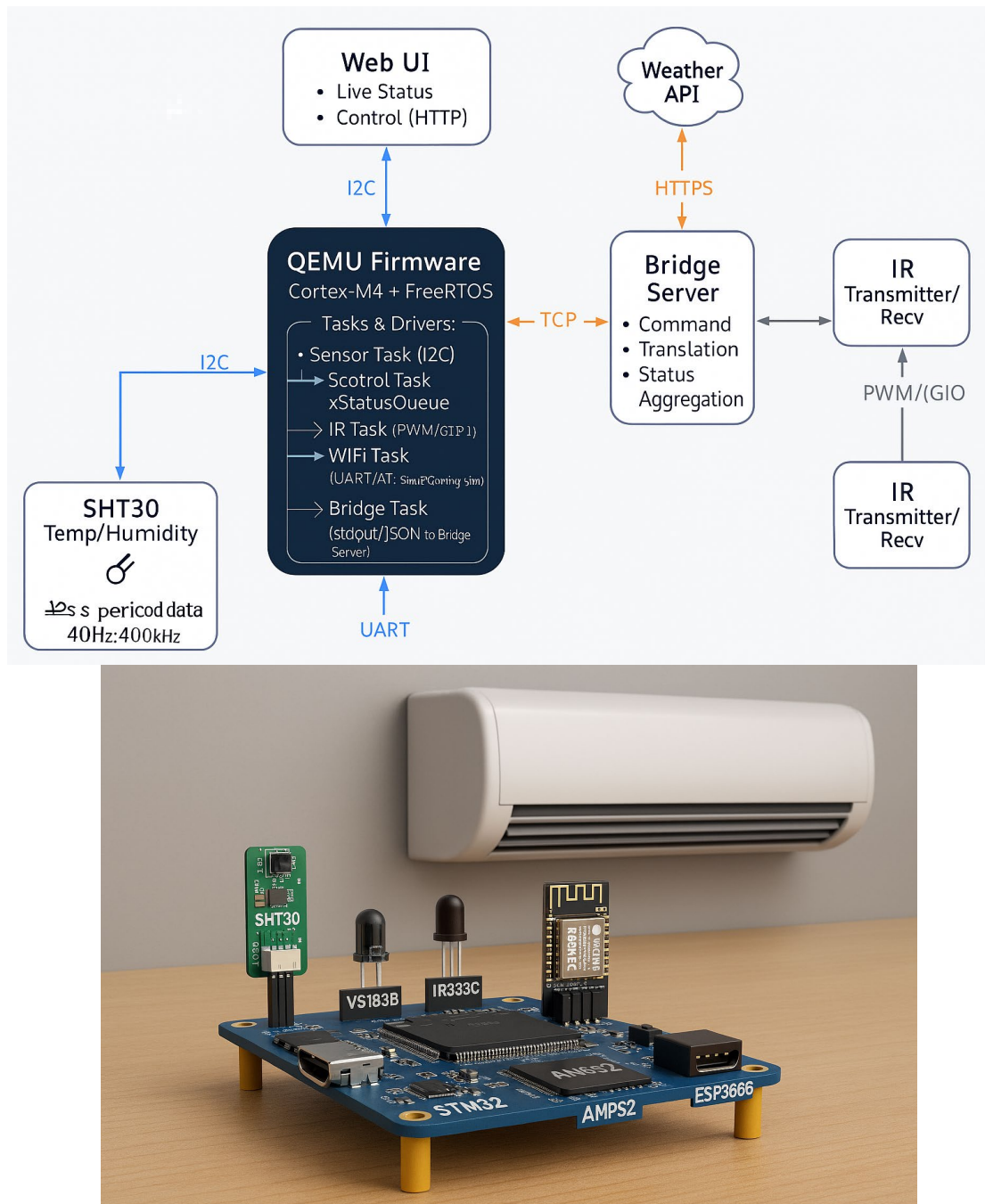


Figure 1 Architecture

3.2 Hardware and Simulation Environment

- **Target Platform:** ARM Cortex-M4 (STM32F407) emulated on QEMU MPS2-AN386.
- **Clock:** 25 MHz system clock.
- **Memory:** 256 KB SRAM + 4 MB Flash.
- **Peripherals:**
 - **SHT30 Sensor:** Temperature and humidity monitoring via I2C (400 kHz).
 - **IR333C IR Transmitter/Vs1838B Receiver:** NEC protocol-based AC control using PWM and GPIO.
 - **ESP8266 Wi-Fi Module:** UART interface for network connectivity.

- **Interfaces:** I2C, UART, PWM, GPIO, HTTP, JSON.

3.3 Software Components

- **FreeRTOS Kernel:** Provides deterministic scheduling and real-time task management.
- **Firmware Modules:**
 - Sensor driver (I2C)
 - IR control driver (PWM + GPIO)
 - Wi-Fi driver (UART + AT commands)
- **Bridge Server:**
 - Acts as middleware between QEMU firmware and web interface.
 - Fetches weather data via Open-Meteo API.
 - Converts commands and status into JSON format.
- **Web Interface:**
 - Displays real-time system status.
 - Allows user control (power, temperature, mode).
 - Supports bilingual UI (Chinese/English).

4. FreeRTOS Architecture and Application

Demonstration

The system runs on **FreeRTOS** under QEMU emulation of Cortex-M4, implementing a **multi-task architecture** to achieve real-time control and modular design. This chapter describes the **task structure**, **priority assignment**, **scheduling strategy**, and **inter-task communication**, followed by an overview of how these tasks cooperate to meet time constraints and enable intelligent air conditioner control.

4.1 Task Overview

The firmware defines **five main tasks**, each with distinct responsibilities:

4.1.1 Sensor Task

- **Function:** Acquire temperature and humidity from SHT30 sensor via I2C.
- **Behavior:** Reads sensor every 2 seconds, converts raw data, and publishes to xSensorQueue.
- **Data Flow:**
SHT30 → Sensor Task → xSensorQueue → Control Task
- **Frequency:** 0.5 Hz (every 2000 ms).
- **Priority:** 4 (**Highest**) to ensure deterministic sampling.
- **Stack:** ~16× minimal size for I2C operations and data conversion.

4.1.2. Control Task

- **Function:** Central logic unit; merges sensor data and user commands, applies AI-based temperature suggestions.
- **Behavior:** Maintains system state (power, mode, target temperature), validates commands, and broadcasts status.
- **Data Flow:**
Sensor Queue → Control Task → xStatusQueue → Bridge Task
- **Frequency:** Status update at ~5 Hz; reacts immediately to new sensor data or

commands.

- **Priority:** 3 for timely decision-making.
- **Stack:** $\sim 8\times$ minimal size for state management and JSON formatting.

4.1.3. IR Task

- **Function:** Infrared control using NEC protocol.
- **Behavior:** Consumes commands from xCommandQueue, encodes IR frames, and drives PWM/GPIO for AC control.
- **Data Flow:**
xCommandQueue $\rightarrow$ IR Task $\rightarrow$ IR Transmitter $\rightarrow$ Air Conditioner
- **Frequency:** Event-driven (on command arrival).
- **Priority:** 3 (same as Control Task) for fast actuation.
- **Stack:** $\sim 8\times$ minimal size for IR encoding and timing control.

4.1.4. WiFi Task

- **Function:** Simulates ESP8266 module, handles UART AT commands and SmartConfig.
- **Behavior:** Maintains connectivity status, processes configuration events, and updates link indicators.
- **Data Flow:**
Bridge Server $\rightarrow$ WiFi Task $\rightarrow$ Control Task (status flags)
- **Frequency:** Periodic checks every 100 ms; event-driven for SmartConfig.
- **Priority:** 2 (moderate urgency).
- **Stack:** Standard size for UART buffer handling.

4.1.5. Bridge Task

- **Function:** Interfaces with external Python bridge server.
- **Behavior:** Serializes system status into JSON for outbound communication and parses inbound commands for dispatch.
- **Data Flow:**
Control Task $\rightarrow$ xStatusQueue $\rightarrow$ Bridge Task $\rightarrow$ Bridge Server $\rightarrow$ Web UI
Web UI $\rightarrow$ Bridge Server $\rightarrow$ Bridge Task $\rightarrow$ xCommandQueue
- **Frequency:** Status broadcast at 5 Hz; command parsing event-driven.
- **Priority:** 1 (**Lowest**) since telemetry can tolerate minor delays.
- **Stack:** $\sim 16\times$ minimal size for JSON serialization.

4.2 Scheduling and Timing

- **Preemptive, fixed-priority scheduling** ensures deterministic behavior:
 - Sensor Task always meets its 2-second sampling deadline.
 - IR Task responds to commands within **<100 ms**.
 - Control Task maintains a 5 Hz update rate for smooth UI feedback.
- **Stack allocation** verified under QEMU: total memory footprint <32 KB.
- **CPU utilization** remains below 50% under mixed load.

4.3 Inter-Task Communication and Synchronization

Queues

- **xSensorQueue:** Sensor $\rightarrow$ Control / Bridge (environmental data).
- **xCommandQueue:** External $\rightarrow$ IR / Control (user commands).
- **xStatusQueue:** Control $\rightarrow$ Bridge (system status).

Data Flow Summary:

Sensor → xSensorQueue → Control → xStatusQueue → Bridge

External Commands → xCommandQueue → IR

Mutex

- **xSystemMutex** protects shared resources (I2C bus, global structures).
- **Timeout-based acquisition (100 ms)** prevents deadlocks and priority inversion.

4.4 Communication Patterns

- **Producer–Consumer:** Sensor → Control → Bridge; External → IR.
- **Synchronous:** Mutex-protected I2C access and queue-based messaging ensure atomic operations.
- **Asynchronous:** Sensor sampling and status broadcasting run independently of user input; IR commands are event-driven.

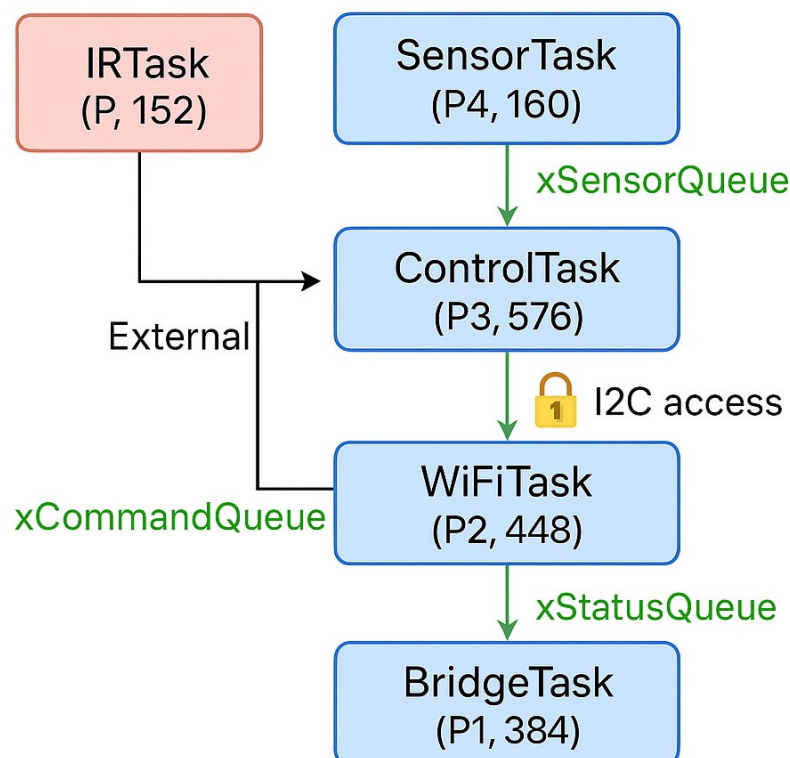


Figure 2 Inter-Task Communication and Synchronization

5. Application Demonstration

This section illustrates how the system operates under real-time constraints, highlighting **multi-tasking behavior**, **task cooperation**, **AI-driven logic**, and **user interface features**.

5.1 Multi-Tasking Behavior Under Time Constraints

The firmware demonstrates **true concurrency** through FreeRTOS preemptive scheduling:

- **Sensor Task** maintains a strict 2-second sampling interval using `vTaskDelayUntil`, unaffected by network or IR operations.

- **IR Task** responds to user commands within **<100 ms**, even during active sensor polling.
- **Control Task** updates system status at **5 Hz**, ensuring smooth feedback for the web interface.
- **Bridge Task** runs opportunistically, broadcasting JSON without blocking higher-priority tasks. This design guarantees **deterministic timing** and **bounded latency**, essential for real-time control.

5.2 Task Cooperation and Communication Flow

Tasks interact through **structured queues** and **mutex synchronization**:

- **Sensor → Control → Bridge**: Sensor data flows via xSensorQueue to Control Task, which processes and forwards consolidated status to Bridge Task through xStatusQueue.
- **External → IR**: User commands from the web interface travel through the Bridge Server and xCommandQueue to IR Task for immediate actuation.
- **WiFi Task** supports SmartConfig and connectivity events, signaling Control Task for status updates. **Communication patterns** include:
 - **Producer–Consumer** for sensor and status data.
 - **Synchronous** mutex-protected I2C access.
 - **Asynchronous** periodic sampling and event-driven IR control.

5.3 AI-Powered Temperature Suggestion Logic

The system integrates **weather data** from the Bridge Server to enhance user experience:

- **Seasonal Adjustment Rules**:
 - Spring/Autumn: ± 1 °C from outdoor temperature.
 - Summer: -4 °C for cooling efficiency.
 - Winter: $+6$ °C for comfort.
- Suggestions are computed by Control Task and displayed on the web interface, enabling **intelligent one-click settings**.

5.4 Web Interface Features

The web interface provides a **bilingual control panel (Chinese/English)** with:

- **Real-time status display**: Indoor temperature, humidity, power state, mode, and AI suggestion.
- **Remote control**: Power toggle, temperature set, mode selection, fan speed adjustment.
- **SmartConfig integration**: Simplified Wi-Fi setup without hardcoded credentials.
- **Scheduling tools**: Timer-based auto-off and ON/OFF scheduling for energy efficiency.
- **Continuous updates**: Status refresh at 5 Hz for near real-time responsiveness.

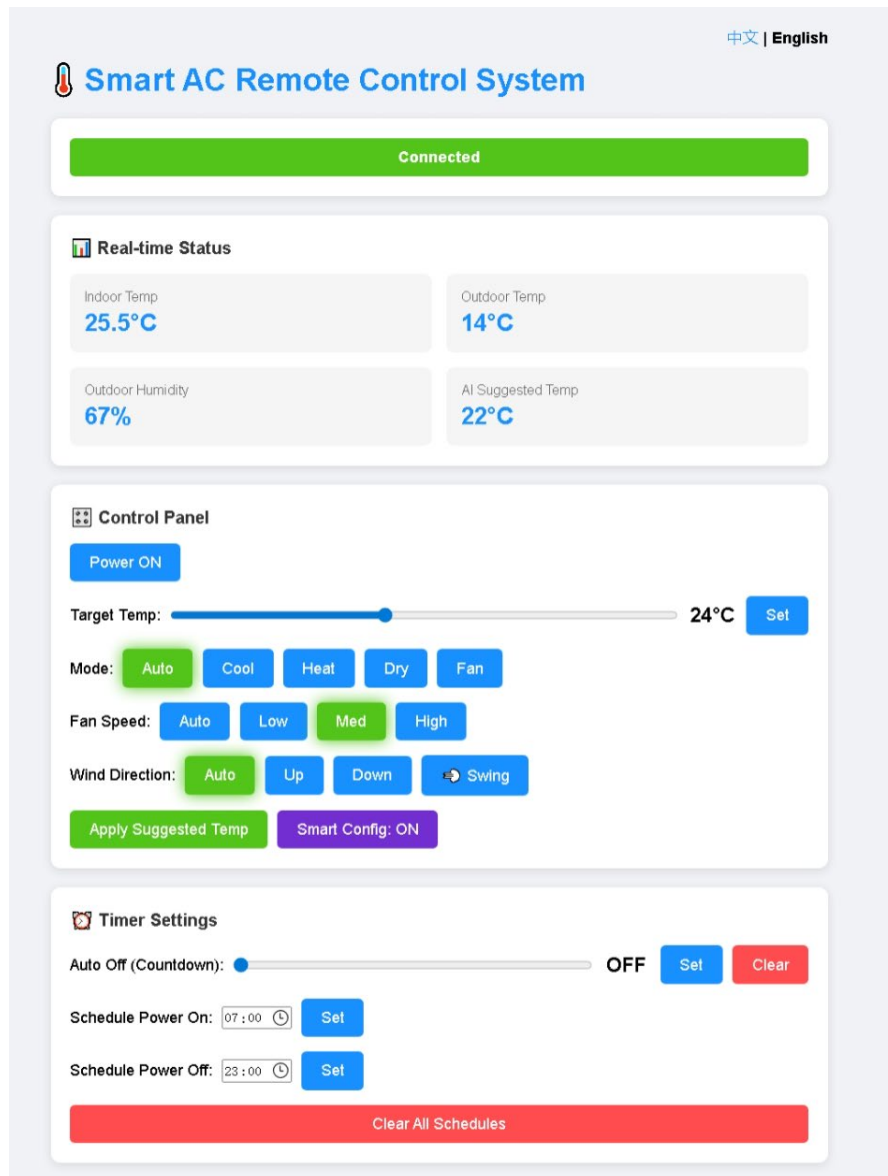


Figure 3 web interface

6. QEMU Simulation Setup

To validate the firmware without physical hardware, the system is emulated on **QEMU**, providing a realistic Cortex-M environment and peripheral simulation.

6.1 Emulation of Cortex-M4 System

- **Target Platform:** ARM Cortex-M4F (STM32F407) using QEMU MPS2-AN386 board model.
- **Configuration:**
 - System clock: 25 MHz.
 - Memory: 256 KB SRAM, 4 MB Flash.
- **FreeRTOS Integration:** The kernel runs natively in QEMU, preserving task priorities, timing, and interrupt behavior for accurate real-time simulation.

6.2 Peripheral Simulation

QEMU provides virtual peripherals to mimic hardware interfaces:

- **I2C:** Simulated SHT30 sensor generates realistic temperature and humidity data

for Sensor Task.

- **UART:** Emulates ESP8266 AT command responses for WiFi Task, including SmartConfig events.
- **PWM/GPIO:** IR Task logs NEC protocol frames instead of driving physical IR LEDs, enabling verification of timing and encoding logic.

6.3 Python Bridge for Weather API Integration

- **Bridge Server** acts as middleware between QEMU firmware and external services:
 - Fetches real-time weather data from **Open-Meteo API**.
 - Computes AI-based temperature suggestions.
 - Translates commands and status into **JSON** for bidirectional communication.

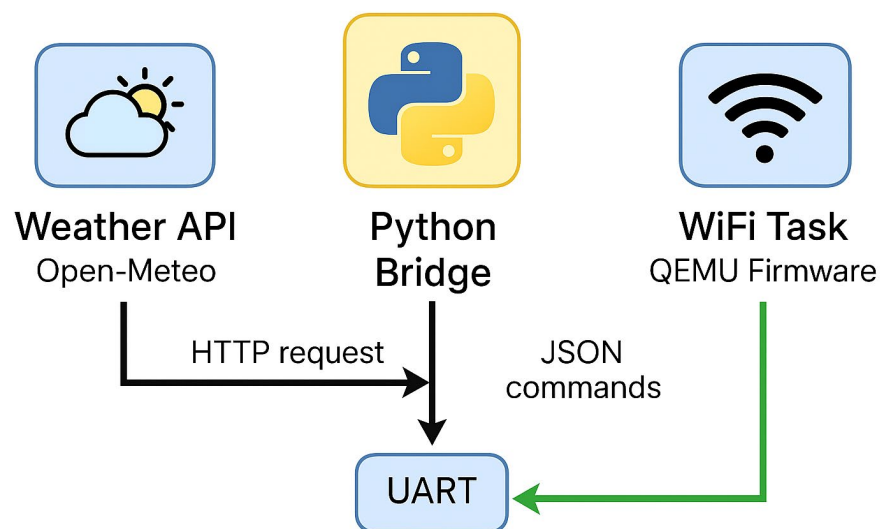


Figure 4 Bridge for Weather API

- **Data Flow:**
 - Firmware → Bridge: Status via stdout (JSON).
 - Bridge → Firmware: Commands via stdin or TCP socket.

7. Conclusion

This project successfully achieved its objectives by implementing a **multi-tasking embedded system** on **FreeRTOS**, emulated on **QEMU Cortex-M4**, and integrating intelligent control for legacy air conditioners. Key accomplishments include:

- **Real-time performance:** Deterministic sensor sampling, bounded command latency, and smooth status updates.
- **Task cooperation:** Five coordinated tasks using queues and mutexes for reliable communication and synchronization.
- **Intelligent features:** AI-powered temperature suggestions based on real-time weather data.
- **User experience:** A bilingual web interface with real-time monitoring, remote

control, and scheduling capabilities.

Educational and Practical Value

The project demonstrates core embedded system principles—real-time scheduling, inter-task communication, hardware abstraction, and system integration—while providing a practical solution for modernizing legacy appliances.

Future Improvements

- **Rust-based implementation** for enhanced safety and concurrency.
 - **Native networking** in QEMU for direct HTTP communication.
 - **Extended AI logic** using machine learning for adaptive comfort control.
 - **Integration with IoT platforms** for cloud-based analytics and remote management.
-

8. References

1. **FreeRTOS Documentation** <https://www.freertos.org>
2. **QEMU Manuals**
<https://www.qemu.org/docs/master>
3. **Open-Meteo Weather API** <https://open-meteo.com>

Acknowledgment of AI Use

I acknowledge the use of Microsoft Copilot (<https://copilot.microsoft.com>) to generate materials(pictures) that I have adapted to include in this submission.

I also acknowledge the use of Microsoft Copilot to help me copy-edit and proofread my writing in this submission.

魏 剑南